Statement of Purpose (SoP)
DSL501: Machine Learning Project

Rohit Raghuwanshi
Roll No: 12341820

1 Project Details

- **Project Title:** Adaptive Teaching Regularization with Class-Aware Knowledge Weighting for Imbalanced and Low-Resource NLP Tasks
- **Code Repo Link:** `https://github.com/rohitraghuwanshi07/Machine-Learning-Project`
- **If Own Idea:** Yes.

This project builds upon Liu et al. (2024) "Learning from Teaching Regularization" (NeurIPS 2024) but introduces **two critical architectural innovations** that address fundamental limitations of the original method. While Liu et al. demonstrated effectiveness on balanced datasets with uniform teaching, their approach suffers from (1) **uniform knowledge transfer** that treats all samples equally regardless of class frequency or sample difficulty, and (2) **static regularization strength** that cannot adapt to varying data characteristics.

Our novel contributions are: (1) **Class-aware knowledge weighting** that dynamically adjusts teaching intensity based on mathematical formulations of class rarity and teacher confidence, and (2) **Lightweight knowledge selection gating** that uses learnable parameters to decide when teacher knowledge is most valuable. These innovations are specifically engineered for imbalanced and low-resource NLP scenarios where traditional uniform approaches fail.

2 Problem Statement

Liu et al. (2024) introduced Teaching Regularization where a teacher model is jointly trained with a student to provide smoother, more generalizable predictions. However, their method has **two critical architectural limitations** that limit real-world applicability:

2.1 Limitation 1: Uniform Knowledge Transfer Problem

The original method uses a fixed regularization approach:

$$\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \cdot \mathrm{KL}(p_{teacher} || p_{student}) \tag{1}$$

where λ is a **static hyperparameter**. This treats all samples identically, which is problematic because:

- **Majority class dominance:** In imbalanced datasets (e.g., 95% normal tweets, 5% hate speech), the model learns primarily from majority classes
- **Minority class neglect:** Rare but important classes receive insufficient teaching attention
- **Sample difficulty ignorance:** Easy and hard samples receive identical treatment

2.2 Limitation 2: Static Regularization Strength

The fixed λ parameter cannot adapt to:

- Varying teacher confidence levels (when teacher is uncertain, more guidance is needed)
- Different sample characteristics (some samples benefit more from teacher guidance)
- Dynamic learning phases (early vs. late training may need different teaching intensities)

2.3 Research Questions

This project addresses these limitations through mathematically principled solutions:

1. **Q1:** Can adaptive class-aware weighting improve minority class performance in imbalanced NLP tasks?
2. **Q2:** How does dynamic regularization strength based on teacher confidence affect model robustness?
3. **Q3:** What is the computational efficiency trade-off of our lightweight architectural modifications?
4. **Q4:** How do these innovations perform in extreme low-resource scenarios (1K-5K samples)?

3 Methodology

Our methodology introduces **Class-Aware Adaptive Teaching Regularization (CATR)**, a novel architecture with two mathematically principled innovations built upon Liu et al. (2024).

3.1 Experimental Setup

3.1.1 Model Architecture

- **Teacher Model:** DistilBERT-base (66M parameters, 6 layers, 768 hidden dimensions)
- **Student Model:** TinyBERT (14.5M parameters, 4 layers, 312 hidden dimensions)
- **Rationale:** Computationally efficient while maintaining meaningful capacity gap for knowledge transfer

3.1.2 Hardware and Software Constraints

- **Primary Hardware:** MacBook Air M2 (8GB RAM, 256GB SSD)
- **Memory Management:** Batch size 16, sequence length 128, FP16 training
- **Backup Compute:** Google Colab free tier for extended experiments
- **Storage Requirements:** ∼50GB total (models, datasets, checkpoints)

3.2 Innovation 1: Class-Aware Knowledge Weighting

3.2.1 Mathematical Foundation

The core innovation replaces Liu et al.'s static λ with an adaptive weighting function $\alpha(x)$ that considers both class frequency and teacher confidence.

Class Frequency Weighting For a dataset with C classes, we define class-specific weights to prioritize underrepresented classes:

$$w_c = \frac{1}{\sqrt{n_c}} \tag{2}$$

where:

- w_c = weight assigned to class c
- n_c = number of training samples in class c
- Square root prevents extreme weight values while ensuring minority classes receive higher attention

Concrete Example: In hate speech detection:

- Normal tweets: $n_{normal} = 10,000 \Rightarrow w_{normal} = \frac{1}{\sqrt{10,000}} = 0.01$
- Hate speech: $n_{hate} = 100 \Rightarrow w_{hate} = \frac{1}{\sqrt{100}} = 0.1$

Thus, hate speech samples receive 10× more teaching attention.

Teacher Confidence Adjustment We incorporate teacher uncertainty to adapt teaching intensity:

$$\text{confidence}(x) = \max(p_{teacher}(x)) \tag{3}$$

$$\text{uncertainty}(x) = 1 - \text{confidence}(x) \tag{4}$$

where $p_{teacher}(x) = [p_1, p_2, ..., p_C]$ is the teacher's probability distribution over classes.

Intuition: When teacher is uncertain (low max probability), student needs more guidance.

Adaptive Weighting Function Combining class frequency and teacher confidence:

$$\alpha(x) = w_c \times \text{uncertainty}(x) = w_c \times (1 - \max(p_{teacher}(x))) \tag{5}$$

Complete Example: For a hate speech sample where teacher predicts $[0.4, 0.1, 0.5]$ (uncertain):

$$\begin{aligned} \alpha(x) &= w_{hate} \times (1 - \max([0.4, 0.1, 0.5])) && (6)\\ &= 0.1 \times (1 - 0.5) && (7)\\ &= 0.1 \times 0.5 = 0.05 && (8) \end{aligned}$$

Modified Teaching Loss The final teaching loss becomes:

$$\mathcal{L}_{teach}(x) = \alpha(x) \times \text{KL}(p_{teacher}(x) || p_{student}(x)) \tag{9}$$

where KL divergence measures the difference between teacher and student probability distributions:

$$\text{KL}(P||Q) = \sum_{i=1}^{C} P(i) \log \left(\frac{P(i)}{Q(i)} \right) \tag{10}$$

3.3 Innovation 2: Lightweight Knowledge Selection Gate

3.3.1 Motivation and Architecture

Not all teacher knowledge is equally valuable for every sample. We introduce a learnable gating mechanism that decides when to prioritize teacher guidance versus independent learning.

Gate Function Design The gate takes both teacher and student hidden representations to make informed decisions:

$$h_{combined} = \text{concat}(h_{teacher}, h_{student}) \tag{11}$$

where:

- $h_{teacher} \in \mathbb{R}^{768}$ = teacher's final hidden state
- $h_{student} \in \mathbb{R}^{312}$ = student's final hidden state
- $h_{combined} \in \mathbb{R}^{1080}$ = concatenated representation

Gate Value Computation A lightweight linear transformation followed by sigmoid activation:

$$g(x) = \sigma(W \cdot h_{combined} + b) \tag{12}$$

where:

- $W \in \mathbb{R}^{1080}$ = learnable weight vector
- $b \in \mathbb{R}$ = learnable bias term
- $\sigma(z) = \frac{1}{1+e^{-z}}$ = sigmoid function mapping to $[0, 1]$

Parameter Overhead: Only 1,081 additional parameters (0.0016% increase for DistilBERT).

Selective Teaching Loss The gate value determines the balance between teacher guidance and independent learning:

$$\mathcal{L}_{selective}(x) = g(x) \cdot \mathcal{L}_{teach}(x) + (1 - g(x)) \cdot \mathcal{L}_{standard}(x) \tag{13}$$

where:

- $g(x) \in [0, 1]$ = gate value (0 = ignore teacher, 1 = rely on teacher)
- $\mathcal{L}_{teach}(x)$ = class-aware teaching loss from Innovation 1
- $\mathcal{L}_{standard}(x)$ = standard cross-entropy loss

Interpretation Examples:

- $g(x) = 0.9$: Student relies heavily on teacher (90% teaching, 10% independent)
- $g(x) = 0.2$: Student prefers independent learning (20% teaching, 80% independent)

3.4 Complete CATR Architecture

3.4.1 Final Loss Function

Combining both innovations, our complete loss function is:

$$\mathcal{L}_{CATR} = \mathcal{L}_{task} + \mathcal{L}_{selective} \tag{14}$$

Expanding the selective component:

$$\mathcal{L}_{CATR} = \mathcal{L}_{task} + g(x) \cdot \alpha(x) \cdot \mathrm{KL}(p_{teacher} || p_{student}) \tag{15}$$

$$+ (1 - g(x)) \cdot \mathcal{L}_{standard} \tag{16}$$

3.4.2 Training Algorithm

Algorithm 1 CATR Training Procedure

Input: Training data $\{(x_i, y_i)\}_{i=1}^{N}$, Teacher model T, Student model S
Initialize: Gate parameters W, b randomly
for each batch B **do**
  for each sample $(x, y) \in B$ **do**
    $p_{teacher} = T(x)$, $p_{student} = S(x)$
    $h_{teacher} = T.hidden(x)$, $h_{student} = S.hidden(x)$
    Compute class weight: $w_c = 1/\sqrt{n_c}$
    Compute uncertainty: $\mathrm{unc}(x) = 1 - \max(p_{teacher})$
    Compute adaptive weight: $\alpha(x) = w_c \times \mathrm{unc}(x)$
    Compute gate: $g(x) = \sigma(W \cdot \mathrm{concat}(h_{teacher}, h_{student}) + b)$
    Compute losses: $\mathcal{L}_{teach}, \mathcal{L}_{standard}, \mathcal{L}_{task}$
    Compute final loss: $\mathcal{L}_{CATR}$
  end for
  Update all parameters via backpropagation
end for

3.5 Experimental Design

3.5.1 Dataset Configuration

1. **Balanced Scenario:** IMDB sentiment analysis (25K samples, 12.5K positive/negative)
2. **Imbalanced Scenario:** Davidson hate speech (25K tweets, ∼5% hate, 77% offensive, 18% neither)
3. **Low-Resource Scenarios:** Systematic subsampling to 1K, 2K, 5K samples while preserving class distributions

3.5.2 Baseline Comparisons

- **Standard Fine-tuning:** DistilBERT without any regularization
- **Original Teaching Reg:** Liu et al. (2024) implementation with fixed λ
- **CATR (ours):** Full implementation with both innovations
- **Ablation Studies:** CATR with only class-weighting, CATR with only gating

3.5.3 Evaluation Methodology

- **Primary Metrics:** Accuracy, Macro-F1 (equal weight to all classes), Per-class F1
- **Efficiency Metrics:** Training time, memory usage, inference latency
- **Robustness Analysis:** Performance degradation across different data sizes
- **Statistical Significance:** 5-fold cross-validation with confidence intervals

4 Dataset Details

4.1 IMDB Movie Reviews Dataset

- **Size:** 50K total reviews (using 25K for memory efficiency)
- **Classes:** Binary sentiment (positive/negative)
- **Balance:** Perfect 50-50 distribution
- **Challenge:** Tests our method's effectiveness on balanced data
- **Source:** `https://ai.stanford.edu/~amaas/data/sentiment/`

4.2 Davidson Hate Speech Dataset

- **Size:** 25K labeled tweets
- **Classes:** Hate speech (5%), Offensive but not hate (77%), Neither (18%)
- **Imbalance Ratio:** 15.4:1 (majority:minority)
- **Challenge:** Extreme class imbalance, real-world social media text
- **Source:** `https://github.com/t-davidson/hate-speech-and-offensive-language`

4.3 Low-Resource Simulation

Systematic subsampling while preserving original class distributions:

- **1K samples:** Hate (50), Offensive (770), Neither (180)
- **2K samples:** Hate (100), Offensive (1540), Neither (360)
- **5K samples:** Hate (250), Offensive (3850), Neither (900)

5 Required Resources

5.1 Computational Resources

- **Primary Hardware:** MacBook Air M2
  - CPU: 8-core (4 performance, 4 efficiency)
  - Memory: 8GB unified memory
  - Storage: 256GB SSD
  - Expected training time: 2-4 hours per experiment
- **Backup Compute:** Google Colab free tier
  - GPU: Tesla T4 (when available)
  - Usage for longer experiments or larger batch sizes

5.2 Software Stack

- **Programming Language:** Python 3.9+
- **Deep Learning:** PyTorch 2.0+ with automatic mixed precision
- **NLP Library:** Hugging Face Transformers 4.30+
- **Data Processing:** Pandas, NumPy, Hugging Face Datasets
- **Evaluation:** Scikit-learn, TorchMetrics
- **Visualization:** Matplotlib, Seaborn, Weights & Biases

5.3 Storage and Version Control

- **Code Repository:** GitHub with detailed documentation
- **Model Storage:** Local checkpoints + HuggingFace Hub backup
- **Experiment Tracking:** Weights & Biases for metrics and visualizations
- **Documentation:** LaTeX for technical reports, Markdown for code docs

6 Novelty of Approach

6.1 Core Technical Contributions

6.1.1 Mathematical Innovation 1: Adaptive Class-Aware Weighting

Novel Aspect: First work to mathematically formulate class-frequency-aware teaching regularization.

Technical Advance:

- Replaces static λ with dynamic $\alpha(x) = w_c \times (1 - \max(p_{teacher}(x)))$
- Theoretically grounded: square root weighting prevents extreme values
- Computationally efficient: no additional parameters, minimal overhead

Comparison with Liu et al. (2024):

- **Liu et al.:** Fixed regularization strength for all samples
- **Our method:** Sample-adaptive regularization based on class rarity and teacher confidence

6.1.2 Architectural Innovation 2: Lightweight Knowledge Selection

Novel Aspect: First gating mechanism specifically designed for teaching regularization.

Technical Advance:

- Learnable gate function: $g(x) = \sigma(W \cdot \text{concat}(h_{teacher}, h_{student}) + b)$
- Minimal parameter overhead: 0.0016% increase
- Adaptive knowledge selection: learns when teacher guidance is beneficial

Distinction from Knowledge Distillation:

- Standard KD: Always transfers all knowledge
- Our approach: Selective transfer based on learned relevance

6.2 Application-Domain Contributions

6.2.1 Imbalanced NLP Tasks

Gap Addressed: Liu et al. (2024) only tested on balanced datasets (CIFAR-10, ImageNet).

Our Contribution:

- First systematic evaluation of teaching regularization on class-imbalanced NLP tasks
- Mathematical framework specifically designed for minority class improvement
- Empirical analysis across different imbalance ratios

6.2.2 Low-Resource Scenarios

Practical Importance: Many real-world NLP applications have limited labeled data.
Our Contribution:

- Comprehensive evaluation across data sizes (1K-25K samples)
- Robustness analysis for extreme data scarcity
- Practical guidelines for deployment in resource-constrained environments

6.3 Engineering Contributions

6.3.1 Resource-Efficient Implementation

Innovation: Architecture designed for practical deployment on limited hardware.
Technical Achievements:

- Memory-efficient training: fits in 8GB RAM
- Fast convergence: 2-4 hours on consumer hardware
- Minimal storage: complete setup under 50GB

7 Individual Contribution

Name: Rohit Raghuwanshi
Roll No: 12341820
Complete Individual Responsibility:

1. **Literature Review:** Comprehensive analysis of teaching regularization, knowledge distillation, and class imbalance handling techniques
2. **Mathematical Formulation:** Development of class-aware weighting equations and gating mechanisms
3. **Architecture Design:** Creation of CATR framework with lightweight, efficient components
4. **Implementation:** Full PyTorch implementation optimized for resource-constrained environments
5. **Experimental Design:** Systematic evaluation protocol across multiple scenarios and baselines
6. **Data Management:** Efficient preprocessing and storage solutions for datasets
7. **Evaluation and Analysis:** Comprehensive metric computation, statistical analysis, and result interpretation
8. **Documentation:** Technical report writing, code documentation, and reproducibility guidelines

This is an individual project with complete original contributions in mathematical formulation, architectural design, and empirical evaluation.

8 Expected Outcomes

8.1 Quantitative

- Demonstrate 3-5% improvement in Macro-F1 for imbalanced datasets
- Show consistent gains across low-resource settings (1K-5K samples)
- Measure computational efficiency: <5% training time increase

8.2 Qualitative

- Analysis of class-aware weighting patterns across different imbalance ratios
- Understanding of when knowledge selection gate activates
- Practical guidelines for applying method to new domains

8.3 Deliverables

- **CATR implementation** optimized for resource-constrained environments
- **Comprehensive evaluation** across multiple data sizes and imbalance ratios
- **Efficiency analysis** including training time and memory usage
- **Reproducible codebase** with clear documentation
- **Technical report** with practical implementation guidelines

9 References

- Liu et al., 2024. Learning from Teaching Regularization. *Advances in Neural Information Processing Systems (NeurIPS)*. `https://papers.nips.cc/paper_files/paper/2024/file/01ce1ae7f94d139e4917f9e4425a4f38-Paper-Conference.pdf`
- Hinton, G., Vinyals, O., & Dean, J., 2015. Distilling the Knowledge in a Neural Network. *Neural Information Processing Systems (NeurIPS)*. `https://arxiv.org/abs/1503.02531`